

1.Task和Thread

Task是C#中非常重要的特性，它代表一个可以异步等待的任务，其源代码位于：<https://github.com/microsoft/referencesource/blob/main/mscorlib/system/threading/Tasks/Task.cs>，首先得明确的是Task并不等于Thread，它和Thread存在本质的区别。

1.1 Task和Thread的区别

Thread则是操作系统级别的真实资源，每当创建一个Thread时，操作系统都会为其在内存中划分出1MB的栈空间，当CPU在多个线程之间切换时也需要保存和恢复寄存器状态，没有Thread，代码无法在CPU上运行。

Task的本质是由CLR维护的Promise+状态机，Task的核心字段有以下几个：

```
1 internal volatile int m_stateFlags: 记录当前的任务状态
2 internal object m_action: 要执行的委托代码
3 private volatile object m_continuationObject: 本质是一个链表，存放任务完成后要执行的代码,即await后续的代码
4 internal TaskScheduler m_taskScheduler: 决定任务交给谁去执行
5 internal object m_stateObject: A state object that can be optionally supplied, passed to action
```

当我们使用new或者Task.Run创建Task的时候，底层的过程为：

1. 在Heap上，CLR创建了一个包含以上核心字段的Task对象；
2. Task对象会将自己包装成一个IThreadPoolWorkItem，并放入C#的全局队列里面；
3. 当线程池ThreadPool里面的某个空闲工作线程开始执行Task里面的任务时，会调用其Execute方法，此时Task开始运行；
4. 当执行完毕或者抛出异常时，Task会将m_stateFlags字段更新为完成，然后去遍历m_continuationObject链表，从而继续执行await后续的代码；

```
1 public class Task : IThreadPoolWorkItem, IAsyncResult, IDisposable
2 {
3     //....
4 }
```

总结：Task只是创建了任务，本身无法执行任务，真正去“干活”的则是底层的ThreadPool!

1.2 Thread的使用场景

在现代C#编程中，基于Task+ThreadPool的组合，已经可以解决大部分的并发问题，但对于一些特殊场景仍需要使用Thread来完成：

- STA线程模型

由于C#线程池里面的线程都是MTA（多线程单元模型），因此对于需要在STA（单线程单元）的特性线程里运行的代码，如C++的特定COM组件等，仍然需要手动创建线程，而不能直接使用Task，伪代码如下：

```
1 var t = new Thread(() =>
2 {
3     //...
4 });
5
6 t.SetApartmentState(ApartmentState.STA);
7 t.Start();
```

- Foreground Threads

对于线程池里面的线程，其默认都是后台线程（Background Threads），即当Main方法执行完毕之后，就会强制结束所有后台未完成的线程，如果想创建Foreground Threads，则需要手动创建线程，并将IsBackground属性设置为false

- 精确的控制

使用Task的时候，我们无法控制Task被分配给哪个核心，但是使用Thread的时候，可以完全进行手动控制，对于一些特定场合：例如长期运行的任务，则需要手动去创建Thread来进行精确的控制。

2.StartNew方法

在C#中创建Task时核心方法为Task.Factory.StartNew,其方法签名如下：

```
1 public Task StartNew(Action<object?> action, object? state, CancellationToken
   cancellationToken,
2                               TaskCreationOptions creationOptions, TaskScheduler
   scheduler)
```

```
1 public Task<TResult> StartNew<TResult>(Func<object?, TResult> function, object?
   state, CancellationToken cancellationToken,
2                               TaskCreationOptions creationOptions, TaskScheduler scheduler)
```

- action/function: 一个委托或者是Function，代表要执行的任务；
- state: delegate或者Function要使用的数据；
- cancellationToken: Task的取消令牌；
- creationOptions: 创建Task的选项和行为；
- scheduler: 决定了任务到底由哪个线程以什么样的方式来执行。

2.1 State

我们都知道C#中使用Lambda进行闭包捕获外部变量的时候，本质会在托管堆Heap上new一个隐藏类，在高并发的场景下，会导致GC压力陡增，这就是为什么在for循环的并发中，都要在内部进行局部变量的实例化。

```
1 for(int i = 0; i < 10; i++)
2 {
3     int index = i;
4     // new i
5     Task.Run(() => {
6         Console.WriteLine(index);
7     })
8     // Parallel worker...
9 }
```

这个时候可以将局部变量作为state传递给Task，它会在编译期将Lambda变成一个全局静态方法并缓存，同时将传进去的对象存入Task的m_stateObject字段里面，从而完美的避免了堆分配和GC压力

```
1 public void ProcessData(Document doc)
2 {
3     Task.Factory.StartNew((stateObj) =>
4     {
5         var myDoc = (Document)stateObj!;
6     },
7     doc,
8     CancellationToken.None,
9     TaskCreationOptions.None,
10    TaskScheduler.Default);
11 }
```

同时，当我们传入state参数之后，该对象不仅可以在 Lambda 内部使用，它还被永久暴露在了Task对象的AsyncState属性上，可以在后续代码中方便的使用。

注意：state的参数类型为object?，因此需要避免Boxing问题，如果传递的是引用类型，可以完美避免闭包问题；但如果传递的是值类型，则会出现Boxing问题。

2.2 TaskCreationOptions

TaskCreationOptions决定了创建Task时的行为方式，是一个Enum类型，且有Flags标志，则意味着可以进行位预算叠加，其定义如下：

```
1 [Flags]
2 public enum TaskCreationOptions
3 {
4     /// <summary>
5     /// Specifies that the default behavior should be used.
6     /// </summary>
7     None = 0x0,
8
9     /// <summary>
```

```
10    /// A hint to a <see cref="TaskScheduler">TaskScheduler</see> to schedule a
11    /// task in as fair a manner as possible, meaning that tasks scheduled
    sooner will be more likely to
12    /// be run sooner, and tasks scheduled later will be more likely to be run
    later.
13    /// </summary>
14    PreferFairness = 0x01,
15
16    /// <summary>
17    /// Specifies that a task will be a long-running, course-grained operation.
    It provides a hint to the
18    /// <see cref="TaskScheduler">TaskScheduler</see> that oversubscription may
    be
19    /// warranted.
20    /// </summary>
21    LongRunning = 0x02,
22
23    /// <summary>
24    /// Specifies that a task is attached to a parent in the task hierarchy.
25    /// </summary>
26    AttachedToParent = 0x04,
27
28    /// <summary>
29    /// Specifies that an InvalidOperationException will be thrown if an
    attempt is made to attach a child task to the created task.
30    /// </summary>
31    DenyChildAttach = 0x08,
32
33    /// <summary>
34    /// Prevents the ambient scheduler from being seen as the current scheduler
    in the created task. This means that operations
35    /// like StartNew or ContinueWith that are performed in the created task
    will see TaskScheduler.Default as the current scheduler.
36    /// </summary>
37    HideScheduler = 0x10,
38
39    // 0x20 is already being used in TaskContinuationOptions
40
41    /// <summary>
42    /// Forces continuations added to the current task to be executed
    asynchronously.
```

```

43     /// This option has precedence over
    TaskContinuationOptions.ExecuteSynchronously
44     /// </summary>
45     RunContinuationsAsynchronously = 0x40
46 }

```

- PreferFairness: 按照任务排队的先后顺序来执行，由于线程池底层使用的是Work Stealing算法，会导致LIFO，在要求绝对FIFO的情况下，可以使用该参数，但会增加线程调度的开销；
- LongRunning: 需要长期在后台执行的任务或者很耗时的任务，这种情况下会直接绕过底层线程池，申请一个独立的Thread；
- AttachedToParent: 允许内部任务依赖在自己线程之上，主要用于处理图/树形结构的并行计算；

```

1 public Task ComputeEfficiencyTreeAsync(Node node)
2 {
3     return Task.Factory.StartNew(() =>
4     {
5
6         node.Loss = Calculate(node);
7         foreach (var child in node.Children)
8         {
9             Task.Factory.StartNew(
10                 () => ComputeEfficiencyTreeAsync(child),
11                 CancellationToken.None,
12                 TaskCreationOptions.AttachedToParent,
13                 TaskScheduler.Default);
14         }
15     },
16     CancellationToken.None,
17     TaskCreationOptions.None,
18     TaskScheduler.Default);
19 }
20 await ComputeEfficiencyTreeAsync(rootNode);

```

- DenyChildAttach: 不允许内部任务依赖在自己线程之上，内部所有的Task相互独立；

- `HideScheduler`: 用来阻断自定义调度器的向下传播, 主要用于底层框架中;
- `RunContinuationsAsynchronously`: 和`Channel`的`AllowSynchronousContinuations = false`作用完全一样, 强制要求`await`后面的代码必须扔回线程队重新排队执行, 不允许在当前线程里面直接内联执行。

注意: 对于`DenyChildAttach`模式, 父任务只负责启动, 外层的`await`会被瞬间释放; `AttachedToParent`只有当所有子`Task`都完成之后, 父任务才会变成`Completed`, 对于`Task.Run`其值默认为`DenyChildAttach`, 对于`Task.Factory.StartNew`其值默认为`AttachedToParent`

2.3 TaskScheduler

`TaskScheduler`决定了任务到底由哪个线程以什么样的方式来执行, 其是一个`abstract class`, 有两个核心的静态字段:

- `TaskScheduler.Current`: 将任务派给当前的调度器, 主要用于需要上下文传递的场景;
- `TaskScheduler.Default`: 采用默认的线程池来执行任务;

同时, 我们还可以基于`ConcurrentExclusiveSchedulerPair`来实现自定义的调度器。

2.4 StartNew注意事项

在使用`Task.Factory.StartNew`创建`Task`时需注意以下几个陷阱:

- 孤儿Action

假设我们需要使用`Task.Factory.StartNew`创建一个`async`任务, 可能会写出以下代码:

```
1 var task = Task.Factory.StartNew(async()=>{
2     await Task.Delay(100);
3     return 1;
4     //...
5 });
6
7 await task;
```

这段看似是传入了一个`async Action`, 但是`Task.Factory.StartNew`根本就不认识`async Action`, 它会将传入的方法当成一个普通的`Action`, 所以`Task.Factory.StartNew`实际上返回的是一个`Task<Task<T>>`, 而不是`Task<T>`! 那么`await task`这行代码只会等待外层的

Task执行完成，根本没有等待内部的Task.Delay，当主线程退出后，内部的异步任务就变成了孤儿委托，所以在这里实际上需要await两次才能拿到内部的结果！但如果使用Task.Run，其会自动帮我们调用.Unwrap()进行内部任务解包，不会出现任何问题。

- **TaskScheduler的死锁**

StartNew默认使用的是Current，如果是在WPF或者Winform等有UI的环境中，如果在UI线程里面调用了耗时的StartNew任务，则可能会导致UI界面卡死甚至是死锁，但是Task.Run默认使用了Default即线程池来执行任务，则可以很好的避免这个问题。

- **AttachedToParent陷阱**

如前所述，StartNew默认使用AttachedToParent模式，例如在Web服务里，每个请求都应该是各自独立的，如果任务A附着于任务B上，则可能会造成难以排查的超时甚至是死锁问题。

3.最佳实践

- 在.NetFramework4.7之后，为了避免使用StartNew创建Task时出现的问题和bug，C#提供了一个新的语法糖：Task.Run，其本质也是调用的StartNew，签名大致如下：

```
1 Task.Factory.StartNew(  
2     action,  
3     CancellationTokn.None,  
4     TaskCreationOptions.DenyChildAttach,  
5     TaskScheduler.Default  
6 )
```

在实际场景中，除非对于一些需要定制化和高级应用的场合，否则建议使用Task.Run来创建Task；

- 即使是使用StartNew来进行定制化，大部分情况下也需要将RunContinuationsAsynchronously属性加上，来避免线程卡死。
- 大部分场景都不需要new Thread()，使用Task和ThreadPool可以处理大部分的并发请求；
- 当需要使用闭包来捕获引用类型参数时，可以通过state参数来避免GC压力；